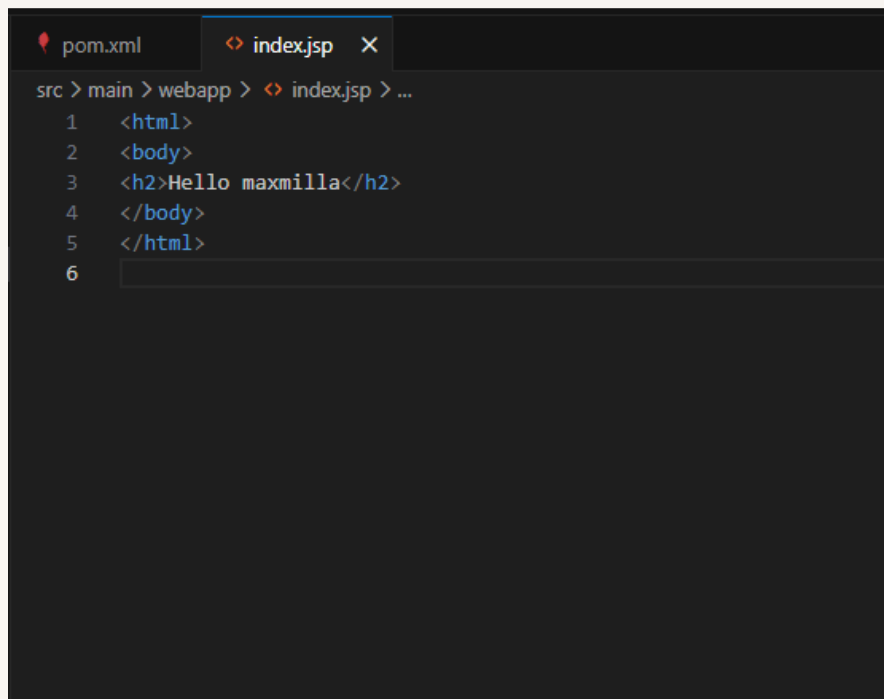


Set Up a Web App Using AWS and VS Code



The image shows a screenshot of the Visual Studio Code editor interface. At the top, there are two tabs: 'pom.xml' and 'index.jsp'. The 'index.jsp' tab is active, showing a file path 'src > main > webapp > index.jsp > ...'. The code in the editor is as follows:

```
1 <html>
2 <body>
3 <h2>Hello maxmilla</h2>
4 </body>
5 </html>
6
```

Introducing Today's Project!

In this project I will be working on setting up a web application in the cloud using Amazon EC2 and VS Code. The goal is to launch a virtual server on AWS, connect to it using SSH from VS Code, and install the required tools such as Java and Maven to generate a basic web application. This project forms the foundation for building a full CI/CD pipeline that will automate the process of building, packaging, and deploying applications in later stages of the DevOps challenge.

This project is part one of a series of DevOps projects where I'm building a full CI/CD pipeline. I'll be working on the next project in the coming days. The next project I'll be tackling is Connect a GitHub Repo with AWS, which I'm really excited about since it will introduce source control integration into the pipeline and bring me one step closer to a fully automated deployment workflow. Documentation for that one will definitely be on the way!

I did this project because I wanted to deepen my understanding of Amazon EC2 and cloud-based development environments, and to refresh my knowledge of the basics involved in setting up and running a project on a virtual machine. It was such an exciting journey to go through the full process of launching a server, connecting to it securely, and getting a web application live in the cloud, all from my local machine.

Key tools and concepts

Services I used were Amazon EC2 for launching and running a virtual server in the cloud, AWS IAM for managing secure user access, and Visual Studio Code (VS Code) as my development environment for connecting to and editing files on my EC2 instance. Key concepts I learnt include how to launch and configure an EC2 instance including setting up key pairs and network/security settings, how SSH works as a secure protocol for remotely accessing a cloud server, how to install and use Apache Maven and Amazon Corretto 8 (Java) to generate a Java web application, and how to connect VS Code to a remote EC2 instance using the Remote SSH extension to edit project files directly on the cloud.

Project reflection

This project took me approximately 4 hours to complete the hands-on work and an additional 1 hour to put together the documentation, bringing the total to around 5 hours overall. The most challenging part was connecting VS Code to the correct SSH host. Since I had several hosts configured, it was tricky identifying which one was the right EC2 instance, and this step ended up consuming a significant amount of my time. Going forward, I'll make sure to clean up and remove any unnecessary SSH hosts to keep things organised and avoid confusion. It was most rewarding to see my project and code actually up and running, knowing that a real virtual server in the cloud was hosting my web application and responding to my changes in real time. That moment made all the troubleshooting worth it!

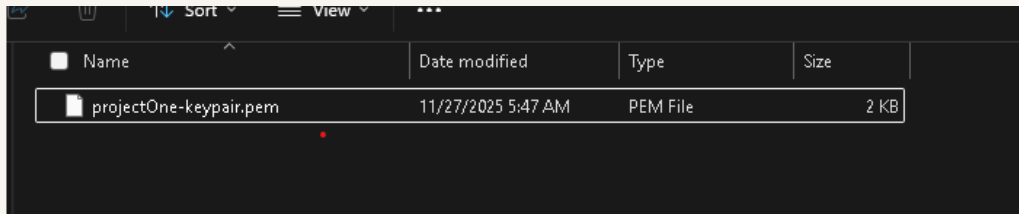
One thing I didn't expect in this project was that I could edit my index.jsp files directly in the terminal using the Nano editor, without needing a full IDE at all. I also didn't expect that a cloud-based project could feel so immediate and local, since changes I made in the terminal were instantly reflected in my VS Code Remote SSH session, showing that both connections were talking to the exact same files on the EC2 instance in real time.

Launching an EC2 instance

What I did in this step

In this step, I am launching a new EC2 instance, creating a key pair for secure SSH access, and configuring the network settings so the server can be reached from the internet. This EC2 instance acts as a virtual server where my web application and development environment will run. The key pair allows me to securely connect to the instance using SSH without relying on passwords, while the network configuration ensures the server can communicate with external systems. This setup establishes the In this step, I first switch my AWS region to one that supports all the services required for the DevOps challenge and is geographically close to me. After selecting the region, I navigate to the Amazon EC2 service and launch a new EC2 instance. This instance acts as a virtual computer in the cloud where my web application and development environment will run. During the setup process, I also foundational infrastructure needed to build, test, and eventually deploy the web application in the cloud.

I started this project by launching an EC2 instance to create a virtual machine in the cloud that I can fully control. Before launching it, I selected an AWS region that supports all the services required for the DevOps challenge so that all my resources remain in the same environment. The EC2 instance provides the compute power needed to run my development environment and web application. It allows me to configure CPU, memory, storage, and network settings just like a normal computer. This instance will serve as the server where I install tools, build my application, and eventually deploy it as part of the CI/CD pipeline.



I also enabled SSH

SSH is a secure protocol used to access and manage a remote server. It verifies that the user connecting has the correct private key that matches the public key stored on the server, ensuring that only authorized users can log in. After authentication, SSH creates an encrypted connection between the user and the server so that all commands and data transferred remain secure. I enabled SSH so I can safely connect to my EC2 instance, run commands, manage files, and control the virtual machine remotely.

Key pairs

Key pair is a security mechanism used to access an EC2 instance. It consists of two parts: a public key that AWS stores on the EC2 server and a private key that you download to your computer.

When you connect to the instance using SSH, the private key verifies your identity and allows secure access to the virtual machine. This method ensures that only the person who owns the private key can log in to the EC2 instance.

Downloaded key pair file

Once I set up my key pair, AWS automatically downloaded a private key file in .pem format to my local computer. This file contains the private half of the key pair and is used to securely connect to my EC2 instance through SSH. I then moved the .pem file into my DevOps folder on my desktop so it is organized and easy to reference when connecting to the server later.

Set up VS Code

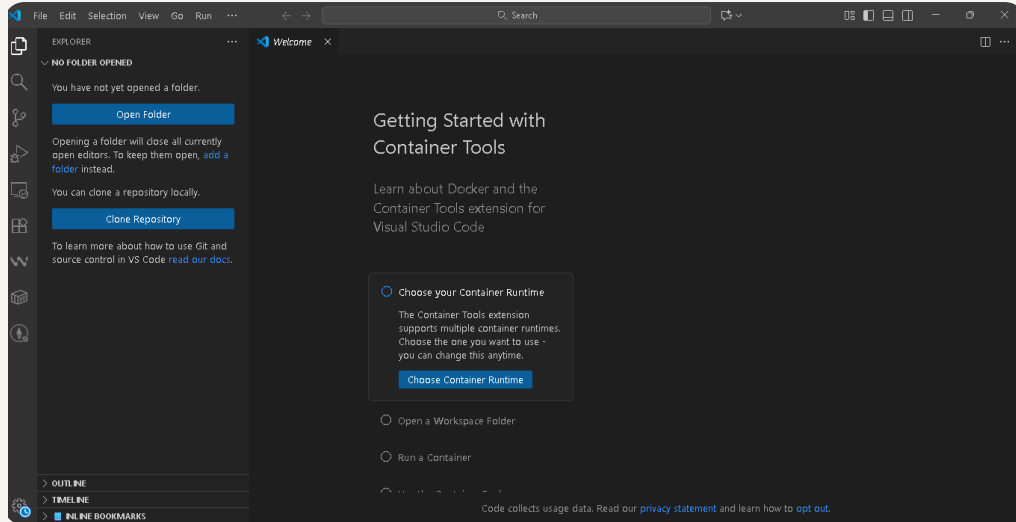
What I did in this step

In this step, I install Visual Studio Code on my computer and set up its terminal so I can communicate with my EC2 instance. I also update the permission settings of my key pair to ensure it can be securely used for SSH authentication. This allows me to safely connect to my EC2 server and use VS Code as my development workspace where I can write, edit, and manage the code for my web application directly from my computer.

What is VS Code?

Visual Studio Code (VS Code) is one of the most popular tools for creating and managing coding projects. It is an Integrated Development Environment (IDE) that allows developers to write, edit, and organize code efficiently. VS Code also includes built-in tools such as a terminal and extensions that make it easier to connect to remote servers like EC2 instances and manage development workflows.

I installed VS Code to help me write and edit code for my web application. I will also use it to open a terminal, connect to my EC2 instance, run commands on the server, and manage my project files while developing and configuring my application in the cloud.



My first terminal commands

The first command I ran in my terminal was `cd ~/Desktop/DevOps` on macOS/Linux or `cd C:\Users\<YourUsername>\Desktop\DevOps` on Windows. The terminal is a text interface that lets you give instructions to your computer using commands instead of clicks. Using the `cd` command, I moved into the DevOps folder where all my project files are stored. This is important because the terminal needs to be in the correct folder to access files like the `.pem` file, which I will use to connect to my EC2 instance.

Updating file permissions

I also updated my private key's permissions by first navigating to the DevOps folder in my terminal to make sure the `.pem` file was accessible. On Windows, I used the `icacls` commands: I ran `icacls "cloud_firstproject.pem" /reset` to remove any existing permission settings, then `icacls "cloud_firstproject.pem" /grant:r maxnj:R` to give my user account read access, and finally `icacls "cloud_firstproject.pem" /inheritance:r` to prevent permissions from being inherited from the parent folder. These steps ensured that only I have access to the file, keeping it secure while making it usable for connecting to my EC2 instance. On Mac/Linux, the equivalent would be `chmod 400 cloud_firstproject.pem`, which sets the file to be readable only by the owner and blocks access for everyone else. This process secures my private key and ensures that SSH connections to my EC2 instance will work properly.

```
hV12P1Ew INdvvP6E9H0S9H0m / D+Rn72/ qhE1n0C1P1v0vxx+mY1n643a1D0X046  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
  
PS C:\Users\maxnj\Desktop\devops\cloud_project-one> icacls "cloud_firstproject.pem" /reset  
>> icacls "cloud_firstproject.pem" /grant:r maxnj:R  
● >> icacls "cloud_firstproject.pem" /inheritance:r  
processed file: cloud_firstproject.pem  
Successfully processed 1 files; Failed processing 0 files  
processed file: cloud_firstproject.pem  
Successfully processed 1 files; Failed processing 0 files  
processed file: cloud_firstproject.pem  
Successfully processed 1 files; Failed processing 0 files  
○ PS C:\Users\maxnj\Desktop\devops\cloud_project-one> 
```

SSH connection to EC2 instance

What I did in this step

In this step, I will connect to my EC2 instance using the VS Code terminal. This allows me to establish a secure session on the remote server so I can run commands, configure the environment, and set up everything needed to deploy my web app. Connecting to the EC2 instance is essential because it gives me direct access to the server's file system and resources, letting me manage software, permissions, and configurations as if I were working locally, but safely on the cloud.

This command required an IPv4 address

A server's IPv4 DNS is the public address that the internet uses to locate and connect to your EC2 instance. It acts like a readable pointer to your server, allowing your computer to find it and establish a direct connection when deploying or accessing your web app.

Maven & Java

What I did in this step

In this step, I will install Apache Maven and Amazon Corretto 8 on my EC2 instance. Apache Maven is a build automation tool that helps manage dependencies, compile code, and package the web app, while Amazon Corretto 8 provides the Java runtime environment needed to run Java applications. Installing and verifying these tools ensures that my EC2 instance has the necessary software to compile, execute, and deploy the web app correctly.

Why I'm using Maven

Apache Maven is a Java build and project management tool that helps developers organize and standardize the structure of their software projects. It automatically downloads and manages external libraries your project depends on, saving time and avoiding version conflicts. Maven also provides archetypes, which are templates that generate the foundational structure for different types of projects, such as web apps, so developers can start coding without worrying about setup details.

placeholder

Maven is used in this project because it automates dependency management, handles compilation and packaging, and enforces a consistent project structure. This makes building, running, and maintaining the Java web app faster, more reliable, and less error-prone, allowing me to focus on developing the actual features of the web app rather than spending time on setup and configuration.

Why I'm using Java

Java is a widely used, object-oriented programming language designed to build a variety of applications, from mobile apps and web applications to large-scale enterprise systems. It is platform-independent, meaning code written in Java can

Create the Application

What I did in this step

In this step, I will run Maven commands in my EC2 terminal to generate a Java web app. Since Maven and Java are already installed, I can use Maven's archetypes to automatically create the project's folder structure, configuration files, and basic templates. This saves time and ensures a consistent setup, allowing me to start developing the web app immediately without manually configuring directories, dependencies, or build settings.

Creating the Java web app

I created the Java web app by running Maven commands directly in my EC2 terminal. Since Java and Maven were already installed on the instance, I used Maven's archetype system to automate project generation. Specifically, I ran `mvn archetype:generate` with parameters that defined the project's group ID (`com.nextwork.app`), artifact ID (`nextwork-web-project`), and archetype type (`maven-archetype-webapp`). The `-DinteractiveMode=false` flag ensured the command executed without stopping for manual input. Maven then automatically generated the project folder structure, configuration files, and template files for the web application. This approach eliminated the need for manual setup, guaranteed a consistent project layout, and allowed me to start development immediately. After execution, I confirmed the setup by checking for the `BUILD SUCCESS` message in the terminal, signaling that the project was created correctly.

Installing Remote - SSH

I installed Remote - SSH to allow VS Code to securely connect to my EC2 instance over SSH so I can work on files, run commands, and manage my application directly on the remote server. This removes the need to manually transfer files and lets me use my local VS Code interface while everything actually runs on the EC2 instance, making development faster and more efficient.

SSH configuration details

The configuration file contained the SSH connection details for my EC2 instance. It included the Host name (a custom alias), the HostName (the EC2 public IPv4 DNS), the User set as ec2-user, and the IdentityFile which pointed to the path of my .pem key file on my local machine. These settings allow VS Code to establish a secure connection to the EC2 instance without re-entering the full SSH command each time.

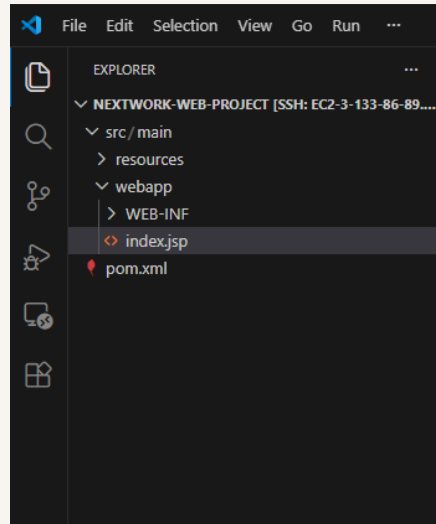
```
[INFO] -----
[INFO] Parameter: basedir, Value: /home/ec2-user
[INFO] Parameter: package, Value: com.projectone.devops
[INFO] Parameter: groupId, Value: com.projectone.devops
[INFO] Parameter: artifactId, Value: projectOneDevops
[INFO] Parameter: packageName, Value: com.projectone.devops
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/ec2-user/projectOneDevops
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.890 s
[INFO] Finished at: 2025-11-29T21:06:11Z
[INFO] Final Memory: 17M/97M
[INFO] -----
```

Create the Application

Exploring the project structure

Using VS Code's file explorer, I could see the nextwork-web-project folder I generated with Maven. Inside it, there are multiple subfolders and files automatically created to structure the web application. This includes the src folder for source code, the pom.xml file that contains Maven's project configuration, and other directories and files that form the backbone of the web app. All folders have expandable subfolders, letting me explore the HTML, CSS, JavaScript, JSP files, and configuration resources that define the app's functionality and appearance.

Two of the project folders created by Maven are src and webapp, which organize the core code and web-specific files for the application. The src folder holds all the source code that defines how the web app behaves and looks. Inside src, the webapp folder specifically contains the web application files such as HTML, CSS, JavaScript, and JSP templates that will be served to users. Additionally, there is a resources folder within src that stores configuration files the app might need, like database connections or other environment settings. This separation ensures the codebase is structured, maintainable, and ready for development.



Using Remote - SSH

What I did in this step

In this step, I connect VS Code to my EC2 instance using a VS Code extension so that I can access and edit the files on the server directly from my IDE. This allows me to use VS Code features like file navigation, code editing, and project management while working on the Java web app running inside the EC2 instance. By doing this, I can easily explore and modify files such as `index.jsp` without relying only on terminal commands.

Updating the web app

The `index.jsp` is a Java Server Page file used in Java web applications. It is similar to an HTML file because it contains markup for displaying web pages, but it can also include Java code to generate dynamic content. This allows the page to change based on user input, database data, or other conditions, unlike static HTML which cannot run Java logic.

I edited `index.jsp` by opening it in VS Code's editor, replacing the placeholder content with my own HTML snippet, including my name in the heading. After making the changes, I saved the file using Command/Ctrl + S, ensuring the editor recognized the updates.

```
pom.xml index.jsp X
src > main > webapp > index.jsp > ...
1 <html>
2 <body>
3 <h2>Hello maxmilla</h2>
4 </body>
5 </html>
6
```


Using nano

Additional improvements

In this secret mission, I will edit `index.jsp` using the terminal instead of an IDE because it teaches me how to modify web app files directly on the server, reinforces understanding of the file structure, and ensures I can make changes even without a graphical editor.

Terminal vs IDE

An alternative to using IDEs is editing code directly in the terminal. To edit `index.jsp`, I ran the command: `bash nano index.jsp` This opened the file in the terminal-based Nano editor, allowing me to navigate, modify, and save the web app's code without leaving the command line.

Compared to using an IDE, editing `index.jsp` in the terminal felt more manual and less visual, since I had to navigate and type without features like syntax highlighting, autocomplete, or file explorers. I'd be more likely to use an IDE if I needed to work quickly, manage multiple files at once, or take advantage of tools that make code editing, debugging, and navigation faster and less error-prone.

Verifying my work

To verify my editing work in the terminal, I returned to the VS Code window connected via Remote-SSH and opened `index.jsp` in the IDE. It was possible to see my changes in VS Code right away because both the terminal and the Remote-SSH session are connected to the same EC2 instance, so edits made in one environment are immediately reflected in the other.

The screenshot shows the VS Code editor with the file 'index.js' open. The code content is as follows:

```
<html>
<body>
<h2>Hello maxmillax</h2>
<p>Im writting this line of code using nano instead of my norlam IDE</p>
</body>
</html>
```

The status bar at the bottom displays various keyboard shortcuts for editing and navigation, such as 'Help', 'Write Out', 'Where Is', 'Cut', 'Execute', 'Location', 'Undo', 'Set Mark', 'To Bracket', 'Exit', 'Read File', 'Replace', 'Justify', 'Go To Line', 'Redo', 'Copy', and 'Where Was'.